

HackerRank Java Programs

Name : Tamil Elakiya S

Reg No : 192524443

1. Question :

HackerRank | Prepare > Java > Introduction Welcome to Java!

Problem

Welcome to the world of Java! In this challenge, we practice printing to stdout.

The code stubs in your editor declare a Solution class and a main method. Complete the main method by copying the two lines of code below and pasting them inside the body of your main method.

```
System.out.println("Hello, World.");
System.out.println("Hello, Java.");
```

Input Format

There is no input for this challenge.

Output Format

You must print two lines of output:

1. Print Hello, World. on the first line.
2. Print Hello, Java. on the second line.

Sample Output

```
Hello, World.
Hello, Java.
```

Submissions

Leaderboard

Code :

```
public class Solution {

    public static void main(String[] args) {
        System.out.println("Hello, World.");
        System.out.println("Hello, Java.");
    }
}
```

Output :

Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1 Hello, World.
2 Hello, Java.
```

Expected Output

```
1 Hello, World.
2 Hello, Java.
```

Download

2. Question :

Task

In this challenge, you must read 3 integers from stdin and then print them to stdout. Each integer must be printed on a new line. To make the problem a little easier, a portion of the code is provided for you in the editor below.

Input Format

There are 3 lines of input, and each line contains a single integer.

Sample Input

```
42
100
125
```

Sample Output

```
42
100
125
```

Code :

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int a = scan.nextInt();
        int b = scan.nextInt();
        int c = scan.nextInt();

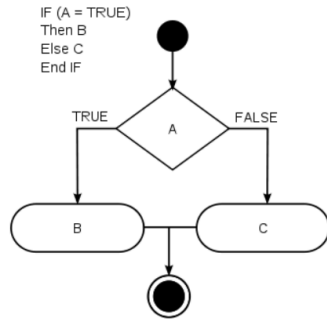
        System.out.println(a);
        System.out.println(b);
        System.out.println(c);
    }
}
```

Output :

Congratulations
You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#) [Next Challenge](#)

✔ Test case 0	Compiler Message	
✔ Test case 1	Success	
✔ Test case 2	Input (stdin)	Download
	<pre>1 42 2 100 3 125</pre>	
	Expected Output	Download
	<pre>1 42 2 100 3 125</pre>	

3 . Question :



Source: Wikipedia

Task

Given an integer, n , perform the following conditional actions:

- If n is odd, print `Weird`
- If n is even and in the inclusive range of 2 to 5, print `Not Weird`
- If n is even and in the inclusive range of 6 to 20, print `Weird`
- If n is even and greater than 20, print `Not Weird`

Complete the stub code provided in your editor to print whether or not n is weird.

Input Format

A single line containing a positive integer, n .

Code :

```
1  import java.io.*;
2  import java.math.*;
3  import java.security.*;
4  import java.text.*;
5  import java.util.*;
6  import java.util.concurrent.*;
7  import java.util.regex.*;
8  import java.util.Scanner;
9
10 public class Solution {
11
12
13
14     private static final Scanner scanner = new Scanner(System.in);
15
16     public static void main(String[] args) {
17         int N = scanner.nextInt();
18         scanner.skip("(\\r\\n|\\[\\n\\r\\u2028\\u2029\\u0085])?");
19         if (N%2!=0){
20             System.out.println("Weird");
21         }else if (N%2==0 && N>1 && N<6){
22             System.out.println("Not Weird");
23         }else if (N%2==0 && N>5 && N<21){
24             System.out.println("Weird");
25         }else if (N%2==0 && N>20){
26             System.out.println("Not Weird");
27         }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0	Input (stdin)	Download
✔ Sample Test case 1	1 3	
	Your Output (stdout)	
	1 Weird	
	Expected Output	Download
	1 Weird	

4 . Question :

In this challenge, you must read an integer, a double, and a String from stdin, then print the values according to the instructions in the Output Format section below. To make the problem a little easier, a portion of the code is provided for you in the editor.

Note: We recommend completing [Java Stdin and Stdout I](#) before attempting this challenge.

Input Format

There are three lines of input:

1. The first line contains an integer.
2. The second line contains a double.
3. The third line contains a String.

Output Format

There are three lines of output:

1. On the first line, print `String`: followed by the unaltered String read from stdin.
2. On the second line, print `Double`: followed by the unaltered double read from stdin.
3. On the third line, print `Int`: followed by the unaltered integer read from stdin.

To make the problem easier, a portion of the code is already provided in the editor.

Code :

```
1  import java.util.Scanner;
2  import java.lang.String;
3
4  public class Solution {
5
6      public static void main(String[] args) {
7          Scanner scan = new Scanner(System.in);
8          int i = scan.nextInt();
9          double d = scan.nextDouble();
10         scan.nextLine();
11         String s = scan.nextLine();
12
13
14         System.out.println("String: " + s);
15         System.out.println("Double: " + d);
16         System.out.println("Int: " + i);
17     }
18 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Sample Test case 1

Input (stdin)

```
1 42
2 3.1415
3 Welcome to HackerRank's Java tutorials!
```

Download

Your Output (stdout)

```
1 String: Welcome to HackerRank's Java tutorials!
2 Double: 3.1415
3 Int: 42
```

Expected Output

```
1 String: Welcome to HackerRank's Java tutorials!
2 Double: 3.1415
```

Download

5 . Question :

Java's `System.out.printf` function can be used to print formatted output. The purpose of this exercise is to test your understanding of formatting output using `printf`.

To get you started, a portion of the solution is provided for you in the editor; you must format and print the input to complete the solution.

Input Format

Every line of input will contain a `String` followed by an integer.

Each `String` will have a maximum of 10 alphabetic characters, and each integer will be in the inclusive range from 0 to 999.

Output Format

In each line of output there should be two columns:

The first column contains the `String` and is left justified using exactly 15 characters.

The second column contains the integer, expressed in exactly 3 digits; if the original input has less than three digits, you must pad your output's leading digits with zeroes.

Code :

```
1 import java.util.Scanner;
2 import java.lang.String;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7         Scanner sc=new Scanner(System.in);
8         System.out.println("=====");
9         for(int i=0;i<3;i++){
10             String s1=sc.next();
11             int x=sc.nextInt();
12             System.out.printf("%-15s%03d%n",s1,x);
13         }
14
15
16         System.out.println("=====");
17
18     }
19 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

Input (stdin)

```
1 java 100
2 cpp 65
3 python 50
```

Your Output (stdout)

```
1 =====
2 java          100
3 cpp           065
4 python        050
5 =====
```

6 . Question :

Task

Given an integer, N , print its first 10 multiples. Each multiple $N \times i$ (where $1 \leq i \leq 10$) should be printed on a new line in the form: $N \times i = \text{result}$.

Input Format

A single integer, N .

Constraints

- $2 \leq N \leq 20$

Output Format

Print 10 lines of output; each line i (where $1 \leq i \leq 10$) contains the *result* of $N \times i$ in the form:

$N \times i = \text{result}$.

Code :

```
1 import java.io.*;
2 import java.math.*;
3 import java.security.*;
4 import java.text.*;
5 import java.util.*;
6 import java.util.concurrent.*;
7 import java.util.regex.*;
8
9
10
11 public class Solution {
12     public static void main(String[] args) throws IOException {
13         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
14
15         int N = Integer.parseInt(bufferedReader.readLine().trim());
16
17         for (int i=1;i<11;i++){
18             int mul=1;
19             mul=N*i;
20             System.out.println(N+" x "+i+" = "+mul);
21         }
22
23         bufferedReader.close();
24     }
25 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

1 2

Your Output (stdout)

```
1 2 x 1 = 2
2 2 x 2 = 4
3 2 x 3 = 6
4 2 x 4 = 8
5 2 x 5 = 10
6 2 x 6 = 12
7 2 x 7 = 14
8 2 x 8 = 16
9 2 x 9 = 18
10 2 x 10 = 20
```

7 . Question :

We use the integers a , b , and n to create the following series:

$$(a + 2^0 \cdot b), (a + 2^0 \cdot b + 2^1 \cdot b), \dots, (a + 2^0 \cdot b + 2^1 \cdot b + \dots + 2^{n-1} \cdot b)$$

You are given q queries in the form of a , b , and n . For each query, print the series corresponding to the given a , b , and n values as a single line of n space-separated integers.

Input Format

The first line contains an integer, q , denoting the number of queries.

Each line i of the q subsequent lines contains three space-separated integers describing the respective a_i , b_i , and n_i values for that query.

Constraints

- $0 \leq q \leq 500$
- $0 \leq a, b \leq 50$
- $1 \leq n \leq 15$

Output Format

For each query, print the corresponding series on a new line. Each series must be printed in order as a single line of n space-separated integers.

Code :

```
1 import java.util.*;
2 import java.io.*;
3
4 class Solution{
5     public static void main(String []argh){
6         Scanner in = new Scanner(System.in);
7         int t=in.nextInt();
8         for(int i=0;i<t;i++){
9             int a = in.nextInt();
10            int b = in.nextInt();
11            int n = in.nextInt();
12            int sum=a;
13            for (int j=0;j<n;j++){
14                sum+=(1 << j)*b;
15                System.out.print(sum + " ");
16            }
17            System.out.println();
18        }
19        in.close();
20    }
21 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Sample Test case 1

Input (stdin)

```
1 2
2 0 2 10
3 5 3 5
```

Your Output (stdout)

```
1 2 6 14 30 62 126 254 510 1022 2046
2 8 14 26 50 98
```

Expected Output

```
1 2 6 14 30 62 126 254 510 1022 2046
2 8 14 26 50 98
```

8. Question :

Java has 8 primitive data types; char, boolean, byte, short, int, long, float, and double. For this exercise, we'll work with the primitives used to hold integer values (byte, short, int, and long):

- A byte is an 8-bit signed integer.
- A short is a 16-bit signed integer.
- An int is a 32-bit signed integer.
- A long is a 64-bit signed integer.

Given an input integer, you must determine which primitive data types are capable of properly storing that input.

To get you started, a portion of the solution is provided for you in the editor.

Reference: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>

Input Format

The first line contains an integer, T , denoting the number of test cases.

Each test case, T , is comprised of a single line with an integer, n , which can be arbitrarily large or small.

Code :

```
6  class Solution{
7      public static void main(String []argh)
8
9
10
11
12      Scanner sc = new Scanner(System.in);
13      int t=sc.nextInt();
14
15      for(int i=0;i<t;i++)
16      {
17
18          try
19          {
20              long x=sc.nextLong();
21              System.out.println(x+" can be fitted in:");
22              if(x>=Byte.MIN_VALUE && x<= Byte.MAX_VALUE)System.out.println("* byte");
23              if(x>=Short.MIN_VALUE && x<=Short.MAX_VALUE)System.out.println("* short");
24              if(x>=Integer.MIN_VALUE && x<=Integer.MAX_VALUE)System.out.println("* int");
25              if(x>=Long.MIN_VALUE && x<=Long.MAX_VALUE)System.out.println("* long");
26          }
27          catch(Exception e)
28          {
29              System.out.println(sc.next()+" can't be fitted anywhere.");
30          }
31      }
32  }
33 }
34 }
```


Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Your Output (stdout)

[illegible]

9. Question :

The challenge here is to read n lines of input until you reach EOF, then number and print all n lines of content.

Hint: Java's `Scanner.hasNext()` method is helpful for this problem.

Input Format

Read some unknown n lines of input from `stdin(System.in)` until you reach EOF; each line of input contains a non-empty String.

Output Format

For each line, print the line number, followed by a single space, and then the line content received as input.

Code :

```

1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6  import java.util.Scanner;
7
8  public class Solution {
9
10     public static void main(String[] args) {
11         Scanner sc=new Scanner(System.in);
12         int n=1;
13         while (sc.hasNext()){
14             String l=sc.nextLine();
15             System.out.println(n + " " + l);
16             n++;
17         }
18         sc.close();
19     }
20 }

```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 Hello world
2 I am a file
3 Read me until end-of-file.
```

Your Output (stdout)

```
1 1 Hello world
2 2 I am a file
3 3 Read me until end-of-file.
```

10. Question :

You are given an integer n , you have to convert it into a string.

Please complete the partially completed code in the editor. If your code successfully converts n into a string s the code will print "Good job".

Otherwise it will print "Wrong answer".

n can range between -100 to 100 inclusive.

Sample Input 0

100

Code :

```
4 import java.util.*; ...
13 String s= Integer.toString(n);
14
15 ✓
16 if (n == Integer.parseInt(s)) {
17     System.out.println("Good job");
18 } else {
19     System.out.println("Wrong answer.");
20 }
21 } catch (DoNotTerminate.ExitTrappedException e) {
22     System.out.println("Unsuccessful Termination!!");
23 }
24 }
25 }
26
27 //The following class will prevent you from terminating the code using exit(0)!
28 class DoNotTerminate {
29
30     public static class ExitTrappedException extends SecurityException {
31
32         private static final long serialVersionUID = 1;
33     }
34
35     public static void forbidExit() {
36         final SecurityManager securityManager = new SecurityManager() {
37             @Override
38             public void checkPermission(Permission permission) {
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

1100

Your Output (stdout)

1Good job

Expected Output

1Good job

11 . Question :

Function Description

Complete the findDay function in the editor below.

findDay has the following parameters:

- int: month
- int: day
- int: year

Returns

- string: the day of the week in capital letters

Input Format

A single line of input containing the space separated month, day and year, respectively, in *MM DD YYYY* format.

Constraints

- $2000 < year < 3000$

Code :

```
1 > import java.io.*; ...
12 import java.time.LocalDate;
13 class Result {
14
15     /*
16      * Complete the 'findDay' function below.
17      *
18      * The function is expected to return a STRING.
19      * The function accepts following parameters:
20      * 1. INTEGER month
21      * 2. INTEGER day
22      * 3. INTEGER year
23      */
24
25     public static String findDay(int month, int day, int year) {
26         java.time.LocalDate date= java.time.LocalDate.of(year , month, day);
27         return date.getDayOfWeek().name();
28     }
29 }
```

Output :

Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)	
1	08 05 2015
Your Output (stdout)	
1	WEDNESDAY
Expected Output	
1	WEDNESDAY

12 . Question : Input Format

A single double-precision number denoting *payment*.

Constraints

- $0 \leq \text{payment} \leq 10^9$

Output Format

On the first line, print US: *u* where *u* is *payment* formatted for US currency.

On the second line, print India: *i* where *i* is *payment* formatted for Indian currency.

On the third line, print China: *c* where *c* is *payment* formatted for Chinese currency.

On the fourth line, print France: *f*, where *f* is *payment* formatted for French currency.

Code :

```
4  import java.math.*;
5  import java.util.regex.*;
6
7  public class Solution {
8
9      public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11         double payment = scanner.nextDouble();
12         scanner.close();
13
14         Locale usLocale = Locale.US;
15         Locale indiaLocale = new Locale("en", "IN");
16         Locale chinaLocale = Locale.CHINA;
17         Locale franceLocale = Locale.FRANCE;
18
19         NumberFormat us=NumberFormat.getCurrencyInstance(usLocale);
20         NumberFormat india=NumberFormat.getCurrencyInstance(indiaLocale);
21         NumberFormat china=NumberFormat.getCurrencyInstance(chinaLocale);
22         NumberFormat france=NumberFormat.getCurrencyInstance(franceLocale);
23
24         System.out.println("US: " + us.format(payment));
25         System.out.println("India: " + india.format(payment));
26         System.out.println("China: " + china.format(payment));
27         System.out.println("France: " + france.format(payment));
28     }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

1 12324.134

Your Output (stdout)

```
1 US: $12,324.13
2 India: Rs.12,324.13
3 China: ¥12,324.13
4 France: 12 324,13 €
```

13 . Question :

The elements of a String are called characters. The number of characters in a String is called the length, and it can be retrieved with the String.length() method.

Given two strings of lowercase English letters, *A* and *B*, perform the following operations:

1. Sum the lengths of *A* and *B*.
2. Determine if *A* is lexicographically larger than *B* (i.e.: does *B* come before *A* in the dictionary?).
3. Capitalize the first letter in *A* and *B* and print them on a single line, separated by a space.

Input Format

The first line contains a string *A*. The second line contains another string *B*. The strings are comprised of only lowercase English letters.

Output Format

There are three lines of output:

For the first line, sum the lengths of *A* and *B*.

For the second line, write Yes if *A* is lexicographically greater than *B* otherwise print No instead.

For the third line, capitalize the first letter in both *A* and *B* and print them on a single line, separated by a space.

Code :

```
4 public class Solution {
5
6     public static void main(String[] args) {
7
8         Scanner sc=new Scanner(System.in);
9         String A=sc.next();
10        String B=sc.next();
11        int l1=A.length();
12        int l2=B.length();
13        System.out.println(l1+l2);
14        if (A.compareTo(B)>0){
15            System.out.println("Yes");
16        }else{
17            System.out.println("No");
18        }
19        String c1=A.substring(0,1).toUpperCase()+A.substring(1);
20        String c2=B.substring(0,1).toUpperCase()+B.substring(1);
21        System.out.println(c1+" "+c2);
22
23        sc.close();
24    }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 hello
2 java
```

Your Output (stdout)

```
1 9
2 No
3 Hello Java
```

14 . Question :

Given a string, s , and two indices, $start$ and end , print a [substring](#) consisting of all characters in the inclusive range from $start$ to $end - 1$. You'll find the String class' [substring method](#) helpful in completing this challenge.

Input Format

The first line contains a single string denoting s .

The second line contains two space-separated integers denoting the respective values of $start$ and end .

Constraints

- $1 \leq |s| \leq 100$
- $0 \leq start < end \leq n$
- String s consists of English alphabetic letters (i.e., $[a - zA - Z]$) only.

Output Format

Print the substring in the inclusive range from $start$ to $end - 1$.

Code :

```
1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6
7 public class Solution {
8
9     public static void main(String[] args) {
10         Scanner in = new Scanner(System.in);
11         String S = in.next();
12         int start = in.nextInt();
13         int end = in.nextInt();
14         String str=S.substring(start,end);
15         System.out.println(str);
16         in.close();
17     }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1  Helloworld
2  3 7
```

Your Output (stdout)

```
1  lowo
```

Expected Output

```
1  lowo
```

15. Question :

Function Description

Complete the getSmallestAndLargest function in the editor below.

getSmallestAndLargest has the following parameters:

- string *s*: a string
- int *k*: the length of the substrings to find

Returns

- string: the string ' + "\n" + ' where and are the two substrings

Input Format

The first line contains a string denoting *s*.

The second line contains an integer denoting *k*.

Constraints

- $1 \leq |s| \leq 1000$
- *s* consists of English alphabetic letters only (i.e., [a-zA-Z]).

Code :

```
1 > import java.util.Scanner; ...
4
5     public static String getSmallestAndLargest(String s, int k) {
6         String smallest = s.substring(0,k);
7         String largest = s.substring(0,k);
8
9         for (int i=1; i<=s.length()-k;i++){
10             String cs=s.substring(i,i+k);
11             if (cs.compareTo(smallest)<0){
12                 smallest=cs;
13             }
14
15             if (cs.compareTo(largest)>0){
16                 largest=cs;
17             }
18         }
19         return smallest + "\n" + largest;
20     }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

```
1 welcometojava
2 3
```

Your Output (stdout)

```
1 ava
2 wel
```

16. Question :

A palindrome is a word, phrase, number, or other sequence of characters which reads the same backward or forward.

Given a string A , print Yes if it is a palindrome, print No otherwise.

Constraints

- A will consist at most 50 lower case english letters.

Sample Input

```
madam
```

Code :

```
1 import java.io.*;
2 import java.util.*;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7
8         Scanner sc=new Scanner(System.in);
9         String A=sc.next();
10        String rev="";
11        for (int i=A.length()-1;i>=0;i--){
12            rev+=A.charAt(i);
13        }
14        if (A.equals(rev)){
15            System.out.println("Yes");
16        }else{
17            System.out.println("No");
18        }
19    }
20 }
```


Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

✓ Sample Test case 1

1 | **madam**

Your Output (stdout)

1 | **Yes**

17 . Question :

Input Format

There are two lines of input. The first line contains B : the breadth of the parallelogram. The next line contains H : the height of the parallelogram.

Constraints

- $-100 \leq B \leq 100$
- $-100 \leq H \leq 100$

Output Format

If both values are greater than zero, then the main method must output the area of the parallelogram. Otherwise, print "java.lang.Exception: Breadth and height must be positive" without quotes.

Code :

```
1  import java.io.*;
2  import java.util.*;
3  import java.util.Scanner;
4
5  public class Solution {
6
7      public static void main(String[] args) {
8          Scanner sc=new Scanner(System.in);
9          int b=sc.nextInt();
10         int h=sc.nextInt();
11         if (b>0 && h>0){
12             int a=b*h;
13             System.out.println(a);
14         }else{
15             System.out.println("java.lang.Exception: Breadth and height must be positive");
16         }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

✓ Sample Test case 1

1 | **1**
2 | **3**

Your Output (stdout)

1 | **3**

18 . Question :

Complete the isAnagram function in the editor.

isAnagram has the following parameters:

- string a: the first string
- string b: the second string

Returns

- boolean: If *a* and *b* are case-insensitive anagrams, return true. Otherwise, return false.

Input Format

The first line contains a string *a*.

The second line contains a string *b*.

Code :

```
1 > import java.util.Scanner; ...
4
5     static boolean isAnagram(String a, String b) {
6         if (a==null || b==null || a.length()!=b.length()){
7             return false;
8         }
9         a=a.toLowerCase();
10        b=b.toLowerCase();
11        int [] f=new int[26];
12        for (int i=0;i<a.length();i++){
13            f[a.charAt(i)-'a']++;
14            f[b.charAt(i)-'a']--;
15        }
16
17        for (int count : f){
18            if (count != 0){
19                return false;
20            }
21        }
22        return true;
23    }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

```
1  anagram
2  margana
```

Your Output (stdout)

```
1  Anagrams
```

19 . Question :

Input Format

A single string, s .

Constraints

- $1 \leq \text{length of } s \leq 4 \cdot 10^5$
- s is composed of any of the following: English alphabetic letters, blank spaces, exclamation points (!), commas (,), question marks (?), periods (.), underscores (_), apostrophes ('), and at symbols (@).

Output Format

On the first line, print an integer, n , denoting the number of tokens in string s (they do not need to be unique). Next, print each of the n tokens on a new line in the same order as they appear in input string s .

Code :

```
1  import java.io.*;
2  import java.util.*;
3
4  public class Solution {
5
6      public static void main(String[] args) {
7          Scanner scan = new Scanner(System.in);
8          String s = scan.nextLine();
9          s=s.trim();
10         if (s.length()==0){
11             System.out.println(0);
12             return;
13         }
14
15         String [] ts = s.split("[^A-Za-z]+");
16         System.out.println(ts.length);
17         for (String t : ts){
18             System.out.println(t);
19         }
20
21         scan.close();
22     }
23 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Sample Test case 1

1 He is a very very good boy, isn't he?

Your Output (stdout)

```
1 10
2 He
3 is
4 a
5 very
6 very
7 good
8 boy
9 isn
```

20. Question :

Input Format

The first line of input contains an integer N , denoting the number of test cases. The next N lines contain a string of any printable characters representing the pattern of a regex.

Output Format

For each test case, print `Valid` if the syntax of the given pattern is correct. Otherwise, print `Invalid`. Do not print the quotes.

Code :

```
1  import java.util.Scanner;
2  import java.util.regex.*;
3
4  public class Solution
5  {
6      public static void main(String[] args){
7          Scanner in = new Scanner(System.in);
8          int testCases = Integer.parseInt(in.nextLine());
9          while(testCases>0){
10             String pattern = in.nextLine();
11             try{
12                 Pattern.compile(pattern);
13                 System.out.println("Valid");
14             }catch (PatternSyntaxException e){
15                 System.out.println("Invalid");
16             }
17             testCases--;
18         }
19         in.close();
20     }
21 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1  3
2  ([A-Z])(.+)
3  [AZ[a-z](a-z)
4  batcatpat(nat
```

Your Output (stdout)

```
1  Valid
2  Invalid
3  Invalid
```

21 . Code :

```
1 > import java.util.regex.Matcher; ...
16 class MyRegex{
17     String n="(\\d{1,2}|(0|1)\\d{2}|2[0-4]\\d|25[0-5])";
18     public String pattern = n+"\\. "+n+"\\. "+n+"\\. "+n;
19 }
20
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 000.12.12.034
2 121.234.12.12
3 23.45.12.56
4 00.12.123.123123.123
5 122.23
6 Hello.IP
```

Your Output (stdout)

```
1 true
2 true
3 true
4 false
```

22. Code :

```
5 public class DuplicateWords {
6
7     public static void main(String[] args) {
8
9         String regex = "\\b(\\w+)(?:\\s+\\1)+\\b";
10        Pattern p = Pattern.compile(regex, Pattern.CASE_INSENSITIVE);
11
12        Scanner in = new Scanner(System.in);
13        int numSentences = Integer.parseInt(in.nextLine());
14
15        while (numSentences-- > 0) {
16            String input = in.nextLine();
17
18            Matcher m = p.matcher(input);
19
20            // Check for subsequences of input that match the compiled pattern
21            while (m.find()) {
22                input = input.replaceAll(m.group(),m.group(1));
23            }
24
25            // Prints the modified sentence.
26            System.out.println(input);
27        }
28    }
29 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

```
1 5
2 Goodbye bye bye world world world
3 Sam went went to to to his business
4 Reya is is the the best player in eye eye game
5 in inthe
6 Hello hello Ab aB
```

Your Output (stdout)

```
1 Goodbye bye world
2 Sam went to his business
3 Reya is the best player in eye game
4 in inthe
```

23 . Code :

```
1 import java.util.Scanner;
2 class UsernameValidator {
3     /*
4      * Write regular expression here.
5      */
6     public static final String regularExpression = "[a-zA-Z][a-zA-Z0-9_]{7,29}$";
7 }
8
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

```
1 8
2 Julia
3 Samantha
4 Samantha_21
5 1Samantha
6 Samantha?10_2A
7 JuliaZ007
8 Julia@007
9 _Julia007
```

Your Output (stdout)

```
1 Invalid
```

24 . Code :

```
7  public class Solution{
8      public static void main(String[] args){
9
10         Scanner in = new Scanner(System.in);
11         int testCases = Integer.parseInt(in.nextLine());
12         String regex = "<([>]+)>([<]+)</\\1>";
13         Pattern pattern = Pattern.compile(regex);
14         while(testCases>0){
15             String line = in.nextLine();
16
17             Matcher matcher=pattern.matcher(line);
18             boolean found=false;
19             while (matcher.find()){
20                 System.out.println(matcher.group(2));
21                 found=true;
22             }
23             if (!found){
24                 System.out.println("None");
25             }
26
27             testCases--;
28         }
29     }
30 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

Download

✓ Sample Test case 1

✓ Sample Test case 2

```
1  4
2  <h1>Nayeem loves counseling</h1>
3  <h1><h1>Sanjay has no watch</h1></h1><par>So wait for a while<par>
4  <Ameesafat codes like a ninja</amee>
5  <SA premium>Imtiaz has a secret crash</SA premium>
```

Your Output (stdout)

```
1  Nayeem loves counseling
2  Sanjay has no watch
3  None
4  Imtiaz has a secret crash
```

25 . Code :

```
1  import java.io.*;
2  import java.math.*;
3  import java.security.*;
4  import java.text.*;
5  import java.util.*;
6  import java.util.concurrent.*;
7  import java.util.regex.*;
8
9
10
11  public class Solution {
12      public static void main(String[] args) throws IOException {
13          BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
14
15          String n = bufferedReader.readLine().trim();
16          BigInteger number = new BigInteger(n);
17          if (number.isProbablePrime(1)){
18              System.out.println("prime");
19          }else{
20              System.out.println("not prime");
21          }
22
23          bufferedReader.close();
24      }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

1 **13**

Your Output (stdout)

1 **prime**

Expected Output

1 **prime**

26. Code :

```
1 import java.util.*;
2 public class test {
3     public static void main(String[] args) {
4         Scanner in = new Scanner(System.in);
5         Deque deque = new ArrayDeque<>();
6         Set set = new HashSet<>();
7         int n = in.nextInt();
8         int m = in.nextInt();
9         int maxUnique = 0;
10
11         for (int i = 0; i < n; i++) {
12             int num = in.nextInt();
13             deque.add(num);
14             set.add(num);
15
16             if (deque.size() == m) {
17                 if (set.size() > maxUnique) {
18                     maxUnique = set.size();
19                 }
20                 if (maxUnique == m) {
21                     break;
22                 }
23                 int removed = (int) deque.removeFirst();
24                 if (!deque.contains(removed)) {
25                     set.remove(removed);
26                 }
27             }
28         }
29     }
30 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 6 3
2 5 3 5 2 3 2
```

Your Output (stdout)

```
1 3
```

Expected Output

```
1 3
```

27. Code :

```
7  ✓ public class Solution {
9  ✓      public static void main(String[] args) {
17 ✓          for (int i=0; i<m; i++){
18              String op = sc.next();
19              int p1 = sc.nextInt();
20              int p2 = sc.nextInt();
21
22 ✓          switch (op){
23 ✓              case "AND":
24                  bitsets[p1].and(bitsets[p2]);
25                  break;
26 ✓              case "OR":
27                  bitsets[p1].or(bitsets[p2]);
28                  break;
29 ✓              case "XOR":
30                  bitsets[p1].xor(bitsets[p2]);
31                  break;
32 ✓              case "FLIP":
33                  bitsets[p1].flip(p2);
34                  break;
35 ✓              case "SET":
36                  bitsets[p1].set(p2);
37                  break;
38              }
39              System.out.println(bitsets[1].cardinality()+" "+bitsets[2].cardinality());
40          }
41          sc.close();
}
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1  5 4
2  AND 1 2
3  SET 1 4
4  FLIP 2 2
5  OR 2 1
```

Your Output (stdout)

```
1  0 0
2  1 0
3  1 1
4  1 2
```

28 . Code :

```
5    import java.math.BigDecimal;...
14
15    // Sort array 's' of size 'n' in descending order
16    Arrays.sort(s, 0, n, new Comparator<String>() {
17        @Override
18        public int compare(String s1, String s2) {
19            BigDecimal b1 = new BigDecimal(s1);
20            BigDecimal b2 = new BigDecimal(s2);
21
22            // Compare b2 to b1 for descending order
23            return b2.compareTo(b1);
24        }
25    });
26 > //Output...
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

Input (stdin)

1	9
2	-100
3	50
4	0
5	56.6
6	90
7	0.12
8	.12
9	02.34
10	000.000

29 . Code :

```
1 import java.util.*;
2 public class Main{
3
4     static Iterator func(ArrayList mylist){
5         Iterator it=mylist.iterator();
6         while(it.hasNext()){
7             Object element = it.next();
8             if(element instanceof String)
9
10                break;
11        }
12        return it;
13
14    }
15    @SuppressWarnings({ "unchecked" })
16    public static void main(String []args){
17        ArrayList mylist = new ArrayList();
18        Scanner sc = new Scanner(System.in);
19        int n = sc.nextInt();
20        int m = sc.nextInt();
21        for(int i = 0;i<n;i++){
22            mylist.add(sc.nextInt());
23        }
24
25        mylist.add("###");
26        for(int i=0;i<m;i++){
27            mylist.add(sc.next());
28        }
29    }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 2 2
2 42
3 10
4 hello
5 java
```

Your Output (stdout)

```
1 hello
2 java
```

30. Code :

```
10
11 ✓ public class Solution {
12 ✓     public static void main(String[] args) throws IOException {
13         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
14
15         int[][] arr = new int[6][6];
16
17         // Read the 6x6 2D array
18 ✓     for (int i = 0; i < 6; i++) {
19         String[] rowItems = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");
20 ✓         for (int j = 0; j < 6; j++) {
21             arr[i][j] = Integer.parseInt(rowItems[j]);
22         }
23     }
24
25     // Initialize max sum to minimum possible integer value
26     int maxHourglassSum = Integer.MIN_VALUE;
27
28     // Iterate through top-left corners of all possible 3x3 hourglasses
29 ✓     for (int i = 0; i <= 3; i++) {
30 ✓         for (int j = 0; j <= 3; j++) {
31             // Sum the 7 elements forming the hourglass
32 ✓             int currentSum = arr[i][j] + arr[i][j+1] + arr[i][j+2]
33                             + arr[i+1][j+1]
34                             + arr[i+2][j] + arr[i+2][j+1] + arr[i+2][j+2];
35         }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 1 1 0 0 0
2 0 1 0 0 0
3 1 1 1 0 0
4 0 9 2 -4 -4 0
5 0 0 0 -2 0 0
6 0 0 -1 -2 -4 0
```

Your Output (stdout)

```
1 13
```

Expected Output

```
1 13
```

31 . Code :

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  public class Solution {
8
9      public static void main(String[] args){
10         Scanner scanner = new Scanner(System.in);
11
12         // Read size of array
13         int n = scanner.nextInt();
14         int[] arr = new int[n];
15
16         // Read array elements
17         for (int i = 0; i < n; i++) {
18             arr[i] = scanner.nextInt();
19         }
20
21         int negativeSubarrayCount = 0;
22
23         // Loop through every possible starting index 'i'
24         for (int i = 0; i < n; i++) {
25             int currentSum = 0;
26
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

```
1  5
2  1 -2 4 -5 1
```

Your Output (stdout)

```
1  9
```

Expected Output

```
1  9
```

32 . Code :

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  public class Solution {
8
9      public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11
12         // Read the number of lines 'n'
13         int n = scanner.nextInt();
14
15         // 2D ArrayList to store rows of varying lengths
16         ArrayList<ArrayList<Integer>> lines = new ArrayList<>();
17
18         // Read each line
19         for (int i = 0; i < n; i++) {
20             int d = scanner.nextInt(); // Number of elements in current line
21             ArrayList<Integer> currentLine = new ArrayList<>();
22
23             for (int j = 0; j < d; j++) {
24                 currentLine.add(scanner.nextInt());
25             }
26
27             lines.add(currentLine);
28         }
29     }
30 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

Input (stdin)

```
1  5
2  5 41 77 74 22 44
3  1 12
4  4 37 34 36 52
5  0
6  3 20 22 33
7  5
8  1 3
9  3 4
10 3 1
11 4 3
12 5 5
```

33 . Code :

```
1  import java.util.*;
2
3  public class Solution {
4
5      public static boolean canWin(int leap, int[] game) {
6          return isSolvable(leap, game, 0);
7      }
8
9      private static boolean isSolvable(int leap, int[] game, int i) {
10         if (i >= game.length) {
11             return true;
12         }
13
14         if (i < 0 || game[i] == 1) {
15             return false;
16         }
17         game[i] = 1;
18         return isSolvable(leap, game, i + leap)
19             || isSolvable(leap, game, i + 1)
20             || isSolvable(leap, game, i - 1);
21     }
22
23     public static void main(String[] args) {
24         Scanner scan = new Scanner(System.in);
25         int q = scan.nextInt();
26         while (q-- > 0) {
27             int n = scan.nextInt();
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1  4
2  5 3
3  0 0 0 0 0
4  6 5
5  0 0 0 1 1 1
6  6 3
7  0 0 1 1 1 0
8  3 1
9  0 1 0
```

Your Output (stdout)

```
1  YES
```


34 . Code :

```
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  public class Solution {
8
9      public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11
12         // Read initial list size N
13         int n = scanner.nextInt();
14
15         // Initialize dynamic List L
16         List<Integer> list = new ArrayList<>();
17         for (int i = 0; i < n; i++) {
18             list.add(scanner.nextInt());
19         }
20
21         // Read number of queries Q
22         int q = scanner.nextInt();
23
24         // Process each query
25         for (int i = 0; i < q; i++) {
26             String queryType = scanner.next();
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1  5
2  12 0 1 78 12
3  2
4  Insert
5  5 23
6  Delete
7  0
```

Your Output (stdout)

```
1  0 1 78 12 23
```

35 . Code :

```
1 //Complete this code or write your own from scratch
2 import java.util.*;
3 import java.io.*;
4
5 class Solution{
6     public static void main(String []argh)
7     {
8         Scanner in = new Scanner(System.in);
9
10        int n = in.nextInt();
11        in.nextLine();
12
13        Map<String, String> phoneBook = new HashMap<>();
14
15
16        for (int i = 0; i < n; i++) {
17            String name = in.nextLine();
18            String phone = in.nextLine();
19            phoneBook.put(name, phone);
20        }
21
22
23        while (in.hasNext()) {
24            String query = in.nextLine();
25
26            if (phoneBook.containsKey(query)) {
27                System.out.println(query + "=" + phoneBook.get(query));
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

```
8 uncle sam
9 uncle tom
10 harry
```

Your Output (stdout)

```
1 uncle sam=99912222
2 Not found
3 harry=12299933
```

Expected Output

```
1 uncle sam=99912222
2 Not found
3 harry=12299933
```

36 . Code :

```
2  class Solution{
4      public static void main(String []argh)
{
8      while (sc.hasNext()) {
9          String input=sc.next();
10         Deque<Character> stack = new ArrayDeque<>();
11         boolean isBalanced = true;
12
13         for (int i = 0; i < input.length(); i++) {
14             char ch = input.charAt(i);
15
16             if (ch == '(' || ch == '{' || ch == '[') {
17                 stack.push(ch);
18             } else {
19                 if (stack.isEmpty()) {
20                     isBalanced = false;
21                     break;
22                 }
23
24                 char top = stack.pop();
25                 if ((ch == ')' && top != '(') ||
26                     (ch == '}' && top != '{') ||
27                     (ch == ']' && top != '[')) {
28                     isBalanced = false;
29                     break;
30                 }
31             }
        }
    }
}
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1  {}()
2  ({()})
3  {}(
4  []
```

Your Output (stdout)

```
1  true
2  true
3  false
4  true
```

37 . Code :

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  public class Solution {
8
9      public static void main(String[] args) {
10         Scanner sc = new Scanner(System.in);
11
12         // Read input strings as BigInteger
13         BigInteger a = new BigInteger(sc.next());
14         BigInteger b = new BigInteger(sc.next());
15
16         // Compute sum and product
17         BigInteger sum = a.add(b);
18         BigInteger product = a.multiply(b);
19
20         // Output results
21         System.out.println(sum);
22         System.out.println(product);
23
24         sc.close();
25     }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

✓ Sample Test case 1

1	1234
2	20

Your Output (stdout)

1	1254
2	24680

38 . Code :

```
1  import java.util.*;
2
3  public class Solution {
4
5      public static void main(String[] args) {
6
7          Scanner scan = new Scanner(System.in);
8          int n = scan.nextInt();
9          int[] a = new int[n];
10         for (int i=0; i<n; i++){
11             a[i] = scan.nextInt();
12         }
13         scan.close();
14
15         // Prints each sequential element in array a
16         for (int i = 0; i < a.length; i++) {
17             System.out.println(a[i]);
18         }
19     }
20 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

1	5
2	10
3	20
4	30
5	40
6	50

Your Output (stdout)

1	10
2	20
3	30
4	40

39. Code :

```
1  ✓
2  import java.io.IOException;
3  import java.lang.reflect.Method;
4
5  class Printer
6  {
7      public <E> void printArray(E[] array){
8          for (E element : array){
9              System.out.println(element);
10         }
11     }
12 }
13
14
15  ✓ public class Solution {
16
17
18      public static void main( String args[] ) {
19          Printer myPrinter = new Printer();
20          Integer[] intArray = { 1, 2, 3 };
21          String[] stringArray = {"Hello", "World"};
22          myPrinter.printArray(intArray);
23          myPrinter.printArray(stringArray);
24          int count = 0;
25
26          for (Method method : Printer.class.getDeclaredMethods()) {
27              String name = method.getName();
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Your Output (stdout)

```
1  1
2  2
3  3
4  Hello
5  World
```

Expected Output

```
1  1
2  2
3  3
4  Hello
5  World
```

40 . Code :

```
1  import java.util.*;
2  class Checker implements Comparator<Player> {
3      @Override
4      public int compare(Player a, Player b) {
5          if (a.score == b.score) {
6              return a.name.compareTo(b.name);
7          }
8          return Integer.compare(b.score, a.score); // Descending order
9      }
10 }
11
12
13 > class Player{ ...
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1  5
2  amy 100
3  david 100
4  heraldo 50
5  aakansha 75
6  aleksa 150
```

Your Output (stdout)

```
1  aleksa 150
2  amy 100
3  david 100
4  aakansha 75
```

41 . Code:

```
25 public class Solution
27 public static void main(String[] args){
42
43 Collections.sort(studentList, new Comparator<Student>() {
44
45 public int compare(Student s1, Student s2) {
46
47 if (s1.getCgpa() != s2.getCgpa()) {
48 return Double.compare(s2.getCgpa(), s1.getCgpa());
49 }
50
51 if (!s1.getFname().equals(s2.getFname())) {
52 return s1.getFname().compareTo(s2.getFname());
53 }
54
55 return Integer.compare(s1.getId(), s2.getId());
56 }
57 });
58
59 for(Student st: studentList){
60 System.out.println(st.getFname());
61 }
62 }
63 }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 5
2 33 Rumpa 3.68
3 85 Ashis 3.85
4 56 Samiha 3.75
5 19 Samara 3.75
6 22 Fahim 3.76
```

Your Output (stdout)

```
1 Ashis
2 Fahim
3 Samara
4 Samiha
```


42 . Code :

HackerRank

Prepare · Java · Object Oriented Programming · Java Inheritance I

Exit Full Screen View

Problem

Finally, we can create a Bird object that can both fly and walk.

```
public class Solution{
    public static void main(String[] args){

        Bird bird = new Bird();
        bird.walk();
        bird.fly();
    }
}
```

Submissions

The above code will print:

```
I am walking
I am flying
```

Leaderboard

This means that a Bird object has all the properties that an Animal object has, as well as some additional unique properties.

The code above is provided for you in your editor. You must add a sing method to the Bird class, then modify the main method accordingly so that the code prints the following lines:

```
I am walking
I am flying
I am singing
```

ons

Change Theme Language Java 7

```
7   class Animal{
10  }
11
12
13  interface Action{
14      void sing();
15  }
16  class Bird extends Animal implements Action{
17      public void fly(){
18          System.out.println("I am flying");
19      }
20      public void sing(){
21          System.out.println("I am singing");
22      }
23  }
24
25  public class Solution{
26
27      public static void main(String args[]){
28
29          Bird bird = new Bird();
30          bird.walk();
31          bird.fly();
32          bird.sing();
33      }
}
```

Line: 20 Col: 12

Output :

Congratulations

You solved this challenge. Would you like to challenge your friends?



Next Challenge

 **Test case 0**

Compiler Message

Success

Expected Output

```
1  I am walking
2  I am flying
3  I am singing
```

Download

43 . Code :

Submissions

Leaderboard

Discussions

Write the following code in your editor below:

1. A class named Arithmetic with a method named add that takes 2 integers as parameters and returns an integer denoting their sum.
2. A class named Adder that inherits from a superclass named Arithmetic.

Your classes should not be public.

Input Format

You are not responsible for reading any input from stdin; a locked code stub will test your submission by calling the add method on an Adder object and passing it 2 integer parameters.

Output Format

You are not responsible for printing anything to stdout. Your add method must return the sum of its parameters.

Sample Output

The main method in the Solution class above should print the following:

```
My superclass is: Arithmetic
42 13 20
```

Change Theme

Language

Java 7

⌕

⋮

```
1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6
7 class Arithmetic {
8     public int add(int a,int b){
9         return a + b;
10    }
11 }
12
13 class Adder extends Arithmetic {
14
15 }
16 public class Solution{
17     public static void main(String []args){
18         // Create a new Adder object
19         Adder a = new Adder();
20
21         // Print the name of the superclass on a new line
22         System.out.println("My superclass is: " + a.getClass().getSuperclass().
23             getName());
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Your Output (stdout)

```
1 My superclass is: Arithmetic
2 42 13 20
```

Expected Output

```
1 My superclass is: Arithmetic
2 42 13 20
```

[Download](#)

44 . Code :

HackerRank

PrepareJavaObject Oriented ProgrammingJava Abstract Class

Exit Full Screen View

Submissions

Leaderboard

Discussions

Tutorial

A Java abstract class is a class that can't be instantiated. That means you cannot create new instances of an abstract class. It works as a base for subclasses. You should learn about Java Inheritance before attempting this challenge.

Following is an example of abstract class:

```
abstract class Book{
    String title;
    abstract void setTitle(String s);
    String getTitle(){
        return title;
    }
}
```

If you try to create an instance of this class like the following line you will get an error:

```
Book new_novel=new Book();
```

You have to create another class that extends the abstract class. Then you can create an instance of the new class.

Notice that setTitle method is abstract too and has no body. That means you must implement the body of that method in the child class.

In the editor, we have provided the abstract Book class and a Main class. In the Main class, we created an instance of a class called MyBook. Your task is to write just the

Change ThemeLanguageJava 7

```
1  import java.util.*;
2  abstract class Book{
3      String title;
4      abstract void setTitle(String s);
5      String getTitle(){
6          return title;
7      }
8  }
9
10 class MyBook extends Book{
11     void setTitle(String s){
12         this.title=s;
13     }
14 }
15
16 > public class Main{...
```

Line: 12 Col: 22

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

[Download](#)

✓ Sample Test case 1

1 A tale of two cities

Your Output (stdout)

1 The title is: A tale of two cities

Expected Output

[Download](#)

1 The title is: A tale of two cities

45 . Code :

Submissions

Leaderboard

Discussions

HackerRank | Prepare > Java > Object Oriented Programming > Java Interface

Exit Full Screen View

A Java interface can only contain method signatures and fields. The interface can be used to achieve polymorphism. In this problem, you will practice your knowledge on interfaces.

You are given an interface AdvancedArithmetic which contains a method signature `int divisor_sum(int n)`. You need to write a class called MyCalculator which implements the interface.

divisorSum function just takes an integer as input and return the sum of all its divisors. For example divisors of 6 are 1, 2, 3 and 6, so divisor_sum should return 12. The value of n will be at most 1000.

Read the partially completed code in the editor and complete it. You just need to write the MyCalculator class only. Your class shouldn't be public.

Sample Input

6

Sample Output

I implemented: AdvancedArithmetic
12

Change Theme Language Java 7

```
1 > import java.util.*; ...
5
6 class MyCalculator implements AdvancedArithmetic {
7     public int divisor_sum(int n){
8         int sum = 0;
9         for (int i=1;i<=n;i++){
10             if (n%i==0){
11                 sum+=i;
12             }
13         }
14         return sum;
15     }
16 }
17
18 > class Solution{ ...
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

✔ Sample Test case 1

✔ Sample Test case 2

Input (stdin)

```
1 6
```

[Download](#)

Your Output (stdout)

```
1 I implemented: AdvancedArithmetic
2 12
```

Expected Output

```
1 I implemented: AdvancedArithmetic
2 12
```

[Download](#)

46. Code :

HackerRank

Prepare > Java > Object Oriented Programming > Java Method Overriding

Exit Full Screen View

Submissions

Leaderboard

Discussions

total

getName method and return a different, subclass-specific string:

```
class Soccer extends Sports{
    @Override
    String getName(){
        return "Soccer Class";
    }
}
```

Note: When overriding a method, you should precede it with the `@Override` annotation. The parameter(s) and return type of an overridden method must be exactly the same as those of the method inherited from the supertype.

Task

Complete the code in your editor by writing an overridden `getNumberOfTeamMembers` method that prints the same statement as the superclass' `getNumberOfTeamMembers` method, except that it replaces `n` with `11` (the number of players on a Soccer team).

Output Format

When executed, your completed code should print the following:

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
Each team has 11 players in Soccer Class
```

Change Theme

Language Java 7

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

> ...

```
1 import java.util.*;
2 class Sports{
3
4     String getName(){
5         return "Generic Sports";
6     }
7
8     void getNumberOfTeamMembers(){
9         System.out.println( "Each team has n players in " + getName() );
10    }
11 }
12
13 class Soccer extends Sports{
14     @Override
15     String getName(){
16         return "Soccer Class";
17     }
18
19     @Override
20     void getNumberOfTeamMembers(){
21         System.out.println("Each team has 11 players in " + getName());
22     }
23 }
24 > ...
```

Line: 23 Col: 1

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Your Output (stdout)

- 1 Generic Sports
- 2 Each team has n players in Generic Sports
- 3 Soccer Class
- 4 Each team has 11 players in Soccer Class

Expected Output

[Download](#)

- 1 Generic Sports
- 2 Each team has n players in Generic Sports
- 3 Soccer Class
- 4 Each team has 11 players in Soccer Class

47 . Code :

HackerRank

PrepareJavaObject Oriented ProgrammingJava Method Overriding 2 (Super Keyword)

Exit Full Screen View

Submissions

Leaderboard

Discussions

When a method in a subclass overrides a method in superclass, it is still possible to call the overridden method using **super** keyword. If you write `super.func()` to call the function `func()`, it will call the method that was defined in the superclass.

You are given a partially completed code in the editor. Modify the code so that the code prints the following text:

Hello I am a motorcycle, I am a cycle with an engine.
My ancestor is a cycle who is a vehicle with pedals.

Change ThemeLanguageJava 7

```
1 import java.util.*;
2 import java.io.*;
3
4 class BiCycle{
5     String define_me(){
6         return "a vehicle with pedals.";
7     }
8 }
9
10 class Motorcycle extends BiCycle{
11     String define_me(){
12         return "a cycle with an engine.";
13     }
14
15     Motorcycle(){
16         System.out.println("Hello I am a motorcycle, I am "+ define_me());
17
18         String temp=super.define_me(); //Fix this line
19
20         System.out.println("My ancestor is a cycle who is "+ temp );...
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Your Output (stdout)

```
1 Hello I am a motorcycle, I am a cycle with an engine.
2 My ancestor is a cycle who is a vehicle with pedals.
```

Expected Output

[Download](#)

```
1 Hello I am a motorcycle, I am a cycle with an engine.
2 My ancestor is a cycle who is a vehicle with pedals.
```

48 . Code :

Problem

Submissions

Leaderboard

The Java instanceof operator is used to test if the object or instance is an instanceof the specified type.

In this problem we have given you three classes in the editor:

- Student class
- Rockstar class
- Hacker class

In the main method, we populated an ArrayList with several instances of these classes. The count method calculates how many instances of each type is present in the ArrayList. The code prints three integers, the number of instance of Student class, the number of instance of Rockstar class, the number of instance of Hacker class.

But some lines of the code are missing, and you have to fix it by modifying only 3 lines! Don't add, delete or modify any extra line.

To restore the original code in the editor, click on the top left icon in the editor and create a new buffer.

Sample Input

```
5
Student
Student
Rockstar
```

Change Theme

Language

Java 7

⌕

⋮

```
2
3 class Student{}
4 class Rockstar{}
5 class Hacker{}
6
7 public class InstanceOfTutorial{
8
9     static String count(ArrayList mylist){
10         int a = 0,b = 0,c = 0;
11         for(int i = 0; i < mylist.size(); i++){
12             Object element=mylist.get(i);
13             if(element instanceof Student)
14                 a++;
15             if(element instanceof Rockstar)
16                 b++;
17             if(element instanceof Hacker)
18                 c++;
19         }
20         String ret = Integer.toString(a)+" "+ Integer.toString(b)+" "+ Integer.
21             toString(c);
22         return ret;
23     }
24     public static void main(String []args){
25         ArrayList mylist = new ArrayList();
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

[Download](#)

```
1 5
2 Student
3 Student
4 Rockstar
5 Student
6 Hacker
```

Your Output (stdout)

```
1 3 1 1
```

Expected Output

[Download](#)

```
1 3 1 1
```

49 . Code :

HackerRank

PrepareJavaAdvancedJava Singleton Pattern

Exit Full Screen View

Problem

Submissions

Leaderboard

Tests

"The singleton pattern is a design pattern that restricts the instantiation of a class to one object. This is useful when exactly one object is needed to coordinate actions across the system."
- Wikipedia: [Singleton Pattern](#)

Complete the Singleton class in your editor which contains the following components:

1. A private Singleton non parameterized constructor.
2. A public String instance variable named `str`.
3. Write a static method named `getSingletonInstance` that returns the single instance of the Singleton class.

Once submitted, our hidden Solution class will check your code by taking a String as input and then using your Singleton class to print a line.

Input Format

You will not be handling any input in this challenge.

Output Format

You will not be producing any output in this challenge.

Sample Input

Change ThemeLanguageJava 7

```
1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6 import java.lang.reflect.*;
7
8
9 class Singleton{
10     // 1. Private static variable to hold the single instance
11     private static Singleton instance;
12
13     // 2. Public String instance variable named str
14     public String str;
15
16     // 3. Private non-parameterized constructor
17     private Singleton() {}
18
19     // 4. Static method that returns the single instance
20     public static Singleton getSingleInstance() {
21         if (instance == null) {
22             instance = new Singleton();
23         }
24         return instance;
25     }
26 }
```

Line: 25 Col: 6

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

[Download](#)

```
1 hello world
```

Your Output (stdout)

```
1 Hello I am a singleton! Let me say hello world to you
```

Expected Output

[Download](#)

```
1 Hello I am a singleton! Let me say hello world to you
```


50 . Code :

Problem

Submissions

Leaderboard

Discussions

Exception handling is the process of responding to the occurrence, during computation, of exceptions — anomalous or exceptional conditions requiring special processing — often changing the normal flow of program execution. (Wikipedia)

Java has built-in mechanism to handle exceptions. Using the try statement we can test a block of code for errors. The catch block contains the code that says what to do if exception occurs.

This problem will test your knowledge on try-catch block.

You will be given two integers x and y as input, you have to compute x/y . If x and y are not 32 bit signed integers or if y is zero, exception will occur and you have to report it. Read sample Input/Output to know what to report in case of exceptions.

Sample Input 0:

```
10
3
```

Sample Output 0:

```
3
```

Sample Input 1:

```
10
Hello
```

HackerRank

PrepareJavaException HandlingJava Exception Handling (Try-catch)

Exit Full Screen View

Change Theme

LanguageJava 7

⌕⋮

```
7  public class Solution {
9      public static void main(String[] args) {
10         Scanner sc = new Scanner(System.in);
11
12         try {
13             // Read two 32-bit signed integers
14             int x = sc.nextInt();
15             int y = sc.nextInt();
16
17             // Compute and display division result
18             System.out.println(x / y);
19
20         } catch (InputMismatchException e) {
21             // Catches errors if inputs are strings or floating-point decimals
22             System.out.println("java.util.InputMismatchException");
23
24         } catch (ArithmeticException e) {
25             // Catches division by zero runtime mathematical errors
26             System.out.println(e);
27         } finally {
28             sc.close();
29         }
30     }
31 }
32
```

Line: 32 Col: 5

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

[Download](#)

✓ Sample Test case 1

```
1 10
2 3
```

✓ Sample Test case 2

Your Output (stdout)

✓ Sample Test case 3

```
1 3
```

✓ Sample Test case 4

Expected Output

```
1 3
```

[Download](#)

